Queue:- Fifo

front = ~~0~~ ~~1~~ ~~2~~ ~~3~~ 4

rear = ~~-1~~ ~~0~~ ~~1~~ ~~2~~ 3

10 20 30 40 50

capacity = 5

→ enqueue
→ isFull()
→ dequeue
→ isEmpty()
→ Front()
→ Rear()

arr = queue

0	1	2	3	4
~~10~~	20	30	40	50

f r f

```
int Front()
{
    isEmpty() — -1
    return arr[front];
}
```

peek()

```
Push(val)
{
    top++
    arr[top] = val
}
```

```
enqueue(val)
{
    isFull() → return;
    rear++;
    arr[rear] = val;
}
```

```
isFull()
{
    If (cap == rear+1) ret. true;
    return false;
}
```

```
int Rear()
{
    isEmpty() — ret -1;
    return arr[rear];
}
```

arr =

0	1	2	3	4
~~10~~	~~20~~	~~30~~	~~40~~	

front ~~0~~ rear front

① dequeue()

```
int dequeue()
{
    isEmpty() return -1;
    int temp = arr[front];
    front++;
    return temp;
}
```

```
isEmpty()
{
    return front > rear;
}
```

Circular Queue:

~~10~~ 20 30 40 50
60

front = ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~ 1

rear = ~~-1~~ ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~ 0

size = ~~0~~ ~~1~~ 2 ~~3~~ ~~4~~ 5 ~~4~~ ~~5~~ ~~4~~ ~~3~~ ~~2~~ ~~1~~ 0

capacity = 5

$\frac{-1}{4}$

0	1	2	3	4
~~10~~	~~20~~	~~30~~	~~40~~	~~50~~

f rear

```
isEmpty() {
    size == 0;
}

isFull() {
    size == 0;
}
```

size = 0 1 2 3 4 5 6 5 4 3 2 1 0

front = 0 1 2 3 4 5 0 1 2

rear = -1

0 1 2 3 4 5 0 1

e(10) cqueue =

e(20) ✓

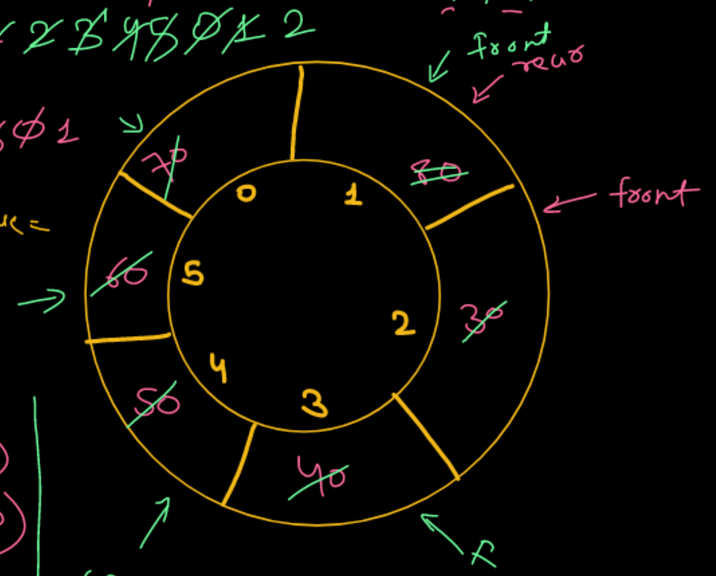
d()

e(30) | e(60) | q()

e(40) | e(70) | q()

d() | e(80) | q()

e(50) | q() | q()



(0+1) % 6 = 1

(1+1) % 6 = 2

(4+1) % 6 = 5

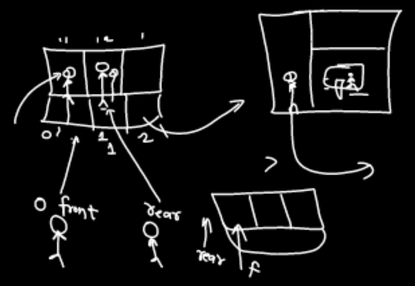
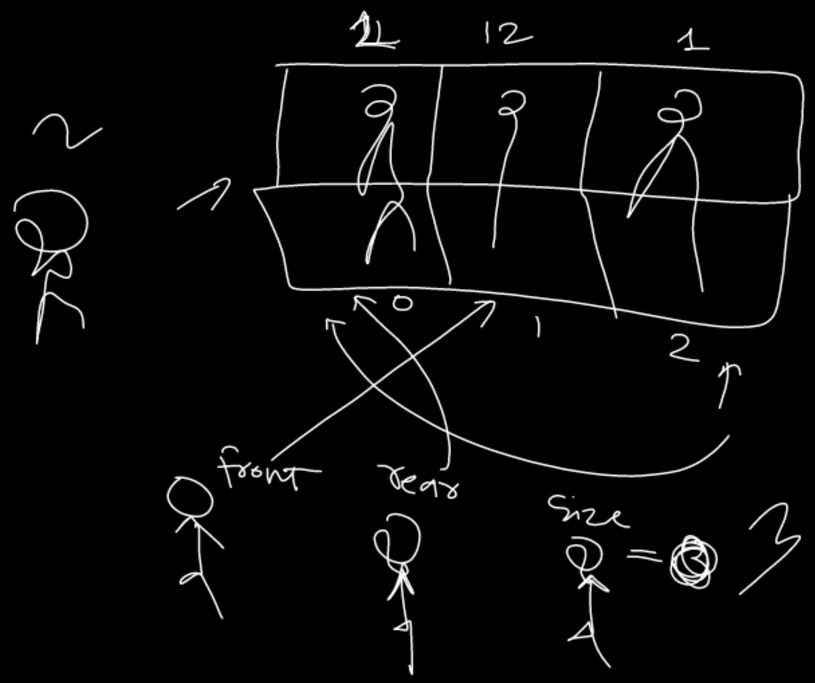
(5+1) % 6

capacity = 6

```
isFull()
{ return size == capacity }

isEmpty()
{ return size == 0; }

int Front()
{ if (isEmpty()) return -1;
  return cqueue[front]; }

int Rear()
{ if (isEmpty())
  return cqueue[rear]; }
```

```
enque(int val)
{ if (isFull()) return;
  rear = (rear + 1) % capacity
  cqueue[rear] = val;
  size++;
}

int dequeue()
{ if (isEmpty()) return -1;
  int item = cqueue[front];
  front = (front + 1) % capacity;
  return item;
}
```

xdf kgfo jpy



front = 0
rear = -1